DAGON UNIVERSITY
DEPARTMENT OF COMPUTER STUDIES

Testing Results and Evaluation Report On Object-Oriented Design and Development of Checkmate Chess Engine

Module No. CS-4107

Presented By

Name	Roll No
မောင်ရဲနိုင်ဦး	စကသ - ၉
မောင်မျိုသော်ပိုင်	စကသ - ၂၀
မဆုထားဆက်နွယ်	စကသ - ၄၀

September, 2026

TABLE OF CONTENTS

CHAPTER 1

Testing Results and Evaluation Report

CHAPTER 1

Testing Results and Evaluation Report

1.1 Introduction

This document presents the testing and evaluation process of the Chat Application, a client-server real-time messaging system developed using C# and .NET 8. The purpose of this testing process is to verify that all features of the Chat Application function correctly, meet the specified requirements, and provide a reliable and user-friendly experience. The testing process includes various types of tests such as functional testing, data storage testing, performance evaluation, and usability evaluation to ensure the overall quality and stability of the system.

1.2 Testing Process Overview

The testing process for the Chat Application was a systematic approach to verify the correctness, reliability, and performance of the developed system. The main objectives of the testing process were to identify and resolve any defects, ensure that all features operate according to the specified requirements, and confirm that the system provides a stable and user-friendly environment for real-time communication.

The testing process involved several key activities, including test planning, test case design, test execution, defect tracking, and result analysis. Test cases were designed to cover all major functions of the Chat Application, including user registration, user authentication, public and private messaging, online user management, message history, and logout functionality. Each test case was executed carefully to compare the actual system behavior with the expected outcome. When defects were discovered, they were recorded, corrected, and tested again until the expected results were achieved. Both positive test cases, which verify normal system operation, and negative test cases, which evaluate the system's response to invalid inputs or incorrect actions, were included in the testing process.

The successful execution of these test cases demonstrated that the Chat Application project meets its functional requirements and provides reliable performance. Comprehensive testing ensured that the application operates accurately, securely, and consistently, giving users confidence in the overall quality and stability of the system.

1.3 Functional Testing Results

Functional testing was performed to verify that all implemented features of the C# Windows Forms Chat Application system work according to the specified functional requirements. The main purpose of functional testing is to ensure that the system provides accurate responses to user actions and that all modules operate correctly under different testing conditions.

The testing process was carried out by using predefined test cases to evaluate each major function of the chat application. The tested functions included user registration, user authentication, starting a new chat session, sending public messages, sending private messages, receiving and displaying messages, viewing online users, accessing message history, and logout functionality.

During testing, both valid and invalid inputs were provided to verify the reliability and error-handling capability of the system. The application was tested with different scenarios, such as incorrect login information, duplicate username registration, sending messages to offline users, and retrieving historical messages.

The results of functional testing showed that all major features of the chat application system operated successfully according to the expected requirements. The application was able to process user commands, route messages correctly, store and retrieve data from the SQLite database, and provide appropriate feedback messages to users. The successful completion of functional testing confirms that the developed Chat Application system is stable, reliable, and suitable for practical usage.

1.4 Functional Testing Components

The Chat Application's functional testing was divided into individual components to ensure that each module of the system was tested thoroughly. The following sections describe each functional testing component in detail.

1.4.1 User Registration Testing

User Registration Testing was conducted to verify the functionality of the user account creation process in the C# Desktop Chat Application. This testing ensures that new users can register successfully before accessing the chat system. The registration form was tested with different input conditions, including valid user information, empty fields, duplicate usernames, and invalid password formats. The system was checked to confirm that user information was validated correctly and stored successfully in the SQLite Database. The test results showed that the application could create new user accounts, prevent duplicate registrations, and display appropriate validation messages when incorrect information was provided. Therefore, the User Registration module functions correctly according to the system requirements.

1.4.2 User Login Testing

User Login Testing was performed to verify the authentication process of the Chat Application system. The purpose of this testing was to ensure that registered users can access the application using correct username and password information. The login function was tested using valid credentials, incorrect passwords, invalid usernames, and empty input fields. The system successfully compared user information with records stored in the SQLite Database (using BCrypt password verification) and provided suitable responses for each condition. When valid credentials were entered, users were allowed to access the main chat interface, while invalid login attempts were rejected. The testing results confirmed that the Login module provides secure and reliable user authentication.

1.4.3 Public Chat Testing

Public Chat Testing was performed to verify that users can send and receive public messages correctly. This testing focused on checking whether public messages are broadcast to all connected online users, displayed properly in the chat interface with sender information and timestamp, and stored in the SQLite database for future reference. The system was tested by sending multiple public messages from different users to ensure that all recipients received the messages correctly. The results showed that the application successfully sent and displayed public messages, updated the chat interface in real-time, and stored all messages in the database. Therefore, the Public Chat function works effectively and satisfies the required functional specifications.

1.4.4 Private Chat Testing

Private Chat Testing was conducted to verify that users can send and receive private one-on-one messages correctly. The private messaging behavior was tested to ensure that messages are sent only to the intended recipient and not broadcast to other users. The testing process checked whether private messages are displayed in the appropriate chat view, include sender and receiver information, and are stored in the database with the private message flag. The system was also tested for sending private messages to offline users and displaying appropriate notifications. The results showed that all private messages were sent and received accurately, and only the intended users could access the private conversation. This confirms that the private messaging logic in the C# application works correctly.

1.4.5 Online Users List Testing

Online Users List Testing was performed to ensure that the Chat Application system correctly displays the list of currently active users. The testing included different scenarios such as users logging in, users logging out, and abrupt disconnections. The system updated the online users list whenever a user joined or left the chat, and all connected clients received the updated list automatically. Invalid scenarios such as duplicate user entries were prevented, and proper synchronization was maintained across all clients. The testing results demonstrated that the online users list mechanism successfully maintains an accurate and up-to-date list of active users.

1.4.6 Message History Testing

Message History Testing was carried out to verify the ability of the system to retrieve and display previous chat messages. The testing focused on checking whether the application could load public chat history when a user logs in, and retrieve private message history when a private chat is selected. Different scenarios were created to test the accuracy of history retrieval, including messages from different dates and large numbers of historical messages. The system successfully displayed the message history with correct sender information, timestamps, and message content, sorted in chronological order. The testing results confirmed that the implemented message history retrieval works correctly and provides users with access to their previous conversations.

1.4.7 Real-time Messaging Testing

Real-time Messaging Testing was conducted to verify that users can send and receive messages in real-time. The testing checked whether messages are delivered immediately after being sent, without significant delays, and that the chat interface updates automatically when a new message is received. The system was tested with multiple users sending messages simultaneously to ensure that all messages were delivered correctly and in the right order. The results showed that the application successfully delivered messages in real-time, providing a smooth and responsive communication experience. Therefore, the Real-time Messaging feature provides effective instant communication.

1.4.8 User Interface Testing

User Interface Testing was performed to verify that all user interface elements of the Chat Application function correctly. The testing included checking button functionality, form navigation, text input fields, and message display areas. Each UI element was tested to confirm that it responds correctly to user interactions and that the interface layout remains consistent. The system successfully handled all UI events and provided appropriate visual feedback to users. The testing results confirmed that the User Interface module provides a consistent and user-friendly interaction experience.

1.4.9 Connection Testing

Connection Testing was performed to verify that the Chat Application can establish and maintain a stable connection between the client and server. The testing included connecting to the server at the specified IP address and port (8888), handling connection failures, and reconnecting after a disconnection. The system was tested under different network conditions to ensure that it could recover from temporary connection issues. The results showed that the application successfully established and maintained client-server connections, and provided appropriate error messages when connection problems occurred. Therefore, the Connection module provides reliable network communication.

1.4.10 Logout Testing

Logout Testing was conducted to verify that users can safely exit their current session from the Chat Application. The testing focused on checking whether the logout function correctly ends the active user session, notifies the server of the disconnection, and returns the user to the login interface. Different logout scenarios were tested to ensure that user information was cleared properly and the user was removed from the online users list. The results showed that the system successfully logged out users and updated the online user list for all remaining clients. Therefore, the Logout module provides proper session management and improves application reliability.

1.5 Data Storage Testing

Data Storage Testing was performed to verify that the Chat Application can store, manage, and retrieve data correctly from different storage locations. The main purpose of this testing is to ensure that user information, message history, and system settings are stored securely and can be accessed accurately when required.

The C# Desktop Chat Application uses SQLite Database as the main storage system for maintaining important data such as registered user accounts and message history. In addition, the client application may use temporary in-memory storage for maintaining the current online users list during a session.

During testing, different storage operations were evaluated, including data insertion, data retrieval, and data querying. The testing process verified that stored information remains consistent and available during application usage. The results confirmed that the data storage functions work correctly and support reliable operation of the Chat Application system.

1.5.1 Central Storage Testing

Central Storage Testing was conducted to verify the functionality of the main database storage system used by the Chat Application. In this project, SQLite Database acts as the central storage location for managing important application data, including user accounts, authentication information, and message history.

The testing focused on verifying the communication between the C# Windows Forms application and SQLite Database. Different operations such as inserting new user records, storing chat messages, retrieving message history, and querying user information were tested to ensure data accuracy and consistency.

The system was tested under different conditions, including adding new records, accessing existing records, and retrieving stored information after restarting the application. The database tables were checked to confirm that data was stored correctly without duplication or loss.

The test results showed that the application successfully connected with SQLite Database and performed all required database operations correctly. User information and message history were stored and retrieved accurately. Therefore, the Central Storage system provides reliable and secure data management for the Chat Application.

1.5.2 Local Storage Testing

Local Storage Testing was performed to verify the ability of the Chat Application to store and manage temporary data on the user's local computer. Local storage is used for maintaining user preferences, application configurations, and temporary session information that does not require permanent database storage.

The testing process evaluated whether local data could be created, updated, and accessed correctly during application execution. Different scenarios were tested, such as saving the last used server IP address and loading it when the application starts.

The system was also tested after closing and reopening the application to ensure that locally stored information remained available and was loaded correctly. The testing verified that local storage operations did not interfere with the main database functions.

The results showed that the application successfully managed local data and provided fast access to temporary information. The Local Storage function improved application performance and user convenience by maintaining necessary settings and preferences. Therefore, the local storage mechanism works effectively according to the functional requirements of the Chat Application system.

1.6 System Performance Evaluation

System Performance Evaluation was conducted to measure the efficiency, stability, and responsiveness of the C# Windows Forms Desktop Chat Application. The main purpose of this evaluation was to determine whether the system can perform all required operations smoothly under normal usage conditions.

The performance of the system was evaluated based on several factors, including application startup time, response time during user interactions, database access speed, memory usage, and overall system stability. The application was tested by performing different operations such as user login, sending public and private messages, retrieving message history, and viewing online users.

The results showed that the Chat Application responded quickly to user commands and performed chat operations efficiently. The connection between the C# application and SQLite Database worked properly, allowing data to be stored and retrieved without significant delays. The chat interface updated correctly after each user action, and the application maintained stable performance during continuous usage with multiple connected users.

However, system performance may depend on the hardware specifications of the user's computer and the number of connected users. Overall, the performance evaluation confirmed that the developed Chat Application system provides reliable operation and meets the required performance expectations.

1.7 Usability Evaluation

Usability Evaluation was performed to assess the ease of use, user interface design, and overall user experience of the C# Desktop Chat Application. The purpose of this evaluation was to determine whether users can easily understand and operate the system without requiring complex instructions.

The usability of the system was evaluated based on interface design, navigation, user interaction, error messages, and accessibility of main functions. The testing involved checking important features such as user registration, login, sending messages, switching between public and private chat, viewing online users, and logout functions.

The evaluation results showed that the application provides a simple and user-friendly interface. Users can easily navigate between different sections and access required functions through clearly designed buttons and menus. The system provides appropriate feedback messages when users perform incorrect actions, such as entering invalid login information or sending messages to non-existent users.

The chat interface was designed to provide a familiar communication environment, allowing users to understand the messaging flow easily. Overall, the usability evaluation confirmed that the Chat Application provides a convenient and enjoyable experience for users.

1.8 Analysis of Testing Results

The analysis of testing results was conducted to evaluate the overall quality and correctness of the developed Chat Application. Functional testing, data storage testing, performance evaluation, and usability evaluation were performed to ensure that the system satisfies the defined requirements.

Based on the functional testing results, all major modules including user registration, user login, public messaging, private messaging, online users list management, message history retrieval, user interface, connection handling, and logout functions operated successfully. The system was able to handle both valid and invalid user inputs correctly.

The data storage testing results confirmed that the application successfully communicated with the SQLite Database and stored important information such as user accounts and message history accurately. The system also managed local storage information effectively for maintaining user preferences and temporary settings.

The performance and usability evaluation results indicated that the application provides stable operation and a user-friendly interface. Although minor improvements can be made in future versions, the current system successfully achieves its main objectives and provides a functional desktop chat communication environment.

1.9 Conclusion

The development and testing of the C# Windows Forms Desktop Chat Application have successfully achieved the main objectives of the project. The system provides essential chat features including user registration, login authentication, public and private messaging, online user management, message history retrieval, and user logout.

Functional testing and data storage testing confirmed that all implemented modules operate correctly and interact properly with the SQLite Database. Performance evaluation showed that the application provides stable and efficient operation, while usability evaluation confirmed that the system is easy to understand and use.

Although there are some limitations, such as the absence of UDP server discovery and advanced chat features, the current system successfully provides a complete desktop chat communication environment. The project demonstrates the effective use of C# Windows Forms and SQLite Database for developing a reliable desktop application.

In conclusion, the Chat Application meets the required functional and technical specifications and provides a strong foundation for future improvements and additional features.

1.10 Limitations and Future Improvement

Although the C# Windows Forms Desktop Chat Application successfully implements the required features, there are some limitations that can be improved in future development. One limitation is that the current system does not include UDP-based server discovery, requiring users to manually enter the server IP address. Users can connect to the server if they know the IP, but automatic server detection on the local network is not available.

Another limitation is that advanced chat features may require further improvement, such as presence notifications for user join/leave events, user roles and display names, announcement messages for administrators, file attachment support, and user profile editing. The current database system stores user and message information locally, which may limit accessibility from different devices.

Future improvements can include implementing UDP-based server discovery for automatic server detection, adding presence notifications, supporting file attachments, improving graphical design, and integrating cloud database services for better data accessibility. Additional security improvements such as session tokens and enhanced password policies can also be introduced.

These future enhancements would improve the functionality, scalability, and user experience of the Chat Application.